INDIVIDUAL WORK ASSIGNMENT • SIX-WEEK PROJECT

PCI Pipeline

Pavement Crack Detection from UAV Imagery

ASSIGNEE	ROLE	COMMITMENT	TOTAL
Saad Ehsan	**ML Intern (OTU)**	**6 weeks · 10 h/week**	**60 hours**

1. Project purpose

METI is developing a prototype that helps road inspection teams review road conditions from drone (UAV) imagery. Three machine-learning pipelines detect different object groups, and a web platform allows users to upload imagery, run analyses, and review the results on the images.

YOUR MISSION

Build and own the PCI (pavement crack detection) pipeline. For each UAV road image, identify pavement cracks, classify their type, and return image-pixel bounding boxes with confidence scores through the shared API. Your output will be displayed by the web platform.

2. Detection classes and priorities

CLASS	DEFINITION / DETECTION TARGET	PRIORITY
Longitudinal crack	Crack roughly parallel to the road centreline. RDD2022: D00.	CORE
Transverse crack	Crack roughly across the road. RDD2022: D10.	CORE
Alligator crack	Connected cracks forming many small, many-sided pieces (fatigue cracking). RDD2022: D20.	CORE
Block crack	Connected cracks forming larger, approximately rectangular blocks.	STRETCH
Ravelling	Loss of surface stones, causing a rough, pitted texture.	STRETCH

Use ASTM D6433 distress definitions as the class-naming reference. Core classes must be working by Week 6. Consider stretch classes only after core classes are stable.

3. Scope boundary

Included: detecting and classifying the listed pavement crack classes in UAV images, with a confidence value and pixel bounding box for each detection.

Excluded from this phase: PCI score calculation, severity ratings, GPS/map coordinates, real-world crack dimensions, and map visualization.

TECHNICAL CONTRACT

Engineering requirements

4. Inputs, outputs, and inference API

- Input: JPEG or PNG UAV images. Every image must have a unique image ID. Provided capture/camera metadata is passed through unchanged, not used by the model.
- Required detection fields: class, confidence (0 to 1), bounding box in image pixels, image ID, and model version. A segmentation mask is optional.
- Bounding-box convention: [x_min, y_min, x_max, y_max] in pixel coordinates.
- Every ML service exposes the same endpoints: POST /jobs, GET /jobs/{job_id}, GET /jobs/{job_id}/results.
- The service runs in its own Docker container and integrates with the full application through Docker Compose.

Illustrative result body from the project brief (final format is agreed with Afnan and the team in Week 1):

```
{
  "job_id": "job-0012",
  "pipeline": "pci",
  "model_version": "pci-v1.0",
  "image_id": "IMG_0457",
  "detections": [
    { "class": "transverse_crack", "confidence": 0.87, "bbox": [412, 1180, 1630, 1265] },
    { "class": "longitudinal_crack", "confidence": 0.64, "bbox": [905, 220, 990, 1410] }
  ]
}
```

5. Data, modelling and evaluation standards

- Research RDD2022 (including China_Drone) and at least one other crack dataset; document licence, image viewpoint, and class mapping.
- Use project UAV images to build labelled data. Convert public annotations into YOLO or COCO format; share the RDD2022 conversion script with Ali.
- Prepare training, validation and fixed test splits. The fixed test set is established in Week 2 and must never be used for training.
- Report mAP@0.5, precision and recall for each class, plus a confusion matrix and class-level image/object counts.
- Log training settings, dataset versions and evaluation results so runs can be reproduced. Store datasets, annotations and trained model weights in versioned shared project storage.
- Use the shared evaluation code. Document known failure cases with example imagery, including shadows, joints and markings mistaken for cracks.

TECHNOLOGY CHOICE IS OPEN

The brief does not mandate a particular detector, ML framework, or inference web framework. Select an appropriate pretrained detector, justify the choice in your one-page approach note, and coordinate deployment/API decisions with Afnan.

DELIVERY ROADMAP / FIRST HALF

Weeks 1–3 · Define, label, train

WEEK 1

Setup & scope

10 hours

- Review RDD2022 (including China_Drone) and at least one other crack dataset; record licence, UAV/ground viewpoint and class mapping (3 h).
- Inspect project UAV imagery, image sizes, crack visibility, lighting and shadows (2 h).
- Write an ASTM D6433-aligned crack labelling guide with representative examples (2 h).
- Agree a shared RDD2022 conversion script and split work with Ali (1 h).
- Write a one-page technical approach note (1 h); check-in, demo and written update (1 h).

END-OF-WEEK EVIDENCE Dataset survey; agreed class list and labelling guide; one-page approach note.

WEEK 2

Data & baseline

10 hours

- Create the shared RDD2022 conversion script (YOLO or COCO format), mapping D00/D10/D20 to project classes (3 h).
- Label approximately 100 project UAV images for the three core crack classes; create training, validation and fixed test splits (4 h).
- Run a pretrained detector on sample images and produce the agreed JSON output, even if results are rough (2 h).
- Check-in, demo and written update (1 h).

END-OF-WEEK EVIDENCE Versioned dataset v1 with splits; baseline inference output using the shared JSON contract.

WEEK 3

Model v1

10 hours

- Fine-tune the detector on public crack data plus labelled project UAV training images (4 h).
- Evaluate on the fixed UAV test set: mAP@0.5, precision/recall per class and confusion matrix (2 h).
- Label approximately 50 additional images focused on the weakest class (3 h).
- Check-in, demo and written update (1 h).

END-OF-WEEK EVIDENCE Model v1, class-level metrics table, and experiment log (settings, dataset version, results).

Labelling targets are planning estimates, not guaranteed quotas. Confirm adjustments with instructors after inspecting object density per image.

DELIVERY ROADMAP / SECOND HALF

Weeks 4–6 · Integrate, improve, hand over

WEEK 4

Integration

10 hours

- Wrap model inference in a service following the agreed three-endpoint API and create its Dockerfile (4 h).
- With Afnan, test the PCI pipeline inside the web platform against the full project UAV image set (3 h).
- Document failures with example images, such as shadows, pavement joints or markings falsely detected as cracks (2 h).
- Check-in, demo and written update (1 h).

END-OF-WEEK EVIDENCE Containerised PCI service running through the platform; failure-case log with examples.

WEEK 5

Improve & harden

10 hours

- Analyse false positives and missed cracks; select one or two improvements (e.g., tiling large images, adding labels, threshold tuning) (3 h).
- Retrain and compare model v2 with v1 on the same fixed test set (3 h).
- Handle blurred, no-road and very large images (2 h).
- Update dataset/model notes (1 h); check-in, demo and written update (1 h).

END-OF-WEEK EVIDENCE Model v2 with before/after results; explicit decision on block crack and ravelling stretch work.

WEEK 6

Report & handover

10 hours

- Freeze the final model; tag trained weights and dataset version (1 h).
- Write the metrics report covering datasets, method, fixed-test results and limitations (4 h).
- Write the README explaining how to reproduce training and inference (2 h).
- Prepare the PCI section of the joint final presentation (2 h).
- Check-in, demo and written update (1 h).

END-OF-WEEK EVIDENCE Final metrics report; reproducible containerised PCI pipeline and README; PCI demo section.

SCOPE RULE

Do not sacrifice the three core crack classes or reliable integration to pursue optional classes. A smaller, reproducible, honestly evaluated pipeline is preferable to an unfinished expanded model.

EXECUTION CHECKLIST

Handover and ways of working

6. Final acceptance checklist — end of Week 6

☐ Core longitudinal, transverse and alligator crack detection works on the project UAV imagery.

☐ Every detection includes the required class, confidence and pixel bounding box, with image ID and model version.

☐ PCI API responds through the agreed endpoints; the PCI container launches with the full platform in Docker Compose.

☐ Final evaluation uses the fixed UAV test set and reports mAP@0.5, precision, recall, confusion matrix and dataset/class counts.

☐ Model versions, labelled data, training settings and weights are stored in shared versioned project storage.

☐ Failure cases and known limitations are documented; the repository contains reproducible setup/training/inference instructions.

☐ Saad contributes the PCI part of the joint presentation and live demo.

7. Team coordination

COLLABORATOR	WHAT TO COORDINATE
Afnan (platform owner)	Finalize API contract and JSON schema in Week 1; align job requests, responses and integration in Week 4.
Ali (IRI owner)	Create one shared RDD2022 converter; avoid duplicate data-format and evaluation work.
All teammates	Use the shared repo, shared evaluation code, peer-reviewed pull requests and protected main branch.

8. Reporting and working expectations

- Post two short written updates every week: what was completed, what comes next and blockers.
- Show working progress at the weekly team check-in; one hour of each 10-hour week is allotted to check-in and updates.
- Raise blockers immediately rather than waiting for a meeting. Record decisions and their reasoning in the repository.
- Report measured performance honestly. Never present evaluation on training images as test-set performance.

9. Decisions / access to confirm in Week 1

- Access to representative project UAV images and shared dataset/model storage.
- GPU or other training compute availability (raise lack of GPU immediately).
- Final pretrained detector and ML/inference framework selection; document rationale.
- Final image submission, job-status and error payloads with Afnan (the sample JSON is not the final API specification).
- Approval of class/labelling rules, dataset licensing, and any changes to suggested labelling targets.